Computational Vision

Diego Chiola

1 Pixel Basics, Image Types, and Color

1.1 Image Types and Pixel Contents

When we think of digital images, we usually think of standard photographs. However, in computational vision, the value stored inside a pixel depends entirely on what kind of sensor captured the image:

- **Pictorial Digital Images (Photos):** These capture light intensity (yielding black-and-white images) or color.
- **Range Images:** Instead of light, the pixels store **depth information** (how far away an object is from the sensor). This is exactly how tools like Microsoft Kinect or Apple's FaceID operate.
- **Medical Images:** Pixels represent the radiation absorbance level of different tissues (e.g., X-rays or CT scans).
- **Thermal Images:** Pixels capture heat signatures, allowing vision in complete darkness.

1.2 Dynamic Range and Quantization

The **dynamic range** of an image refers to the total number of distinctive values that can possibly occur in the image. Think of this as the "palette" of available shades.

This range is limited by two things: the physical capabilities of the camera sensor and the number of bits (data) we choose to assign to each pixel, a process called quantization.

- **1-bit depth:** The simplest image. A pixel can only be 0 or 1, meaning the image is strictly black and white with no shades of gray.
- **8-bit depth (Gray Level):** This is the standard for grayscale images. We associate 1 byte (which is 8 bits) to each pixel.

Numerical Example: Because computers run on binary, an 8-bit depth means we have 2^8 possible combinations.

$$2^8 = 256 \text{ gray levels.}$$

This means a pixel can hold any integer value from 0 (pure black) to 255 (pure white). A value of 128 would be a perfect mid-tone gray.

1.3 Color Image Processing

To capture color, cameras use three different physical filters (sensitive to Red, Green, and Blue light) to acquire three separate monochrome images. These three layers are then stacked together to form a single RGB image.

- **Full Color (24-bit):** In standard color images, every single pixel is described by a triplet of values: (R, G, B).
- Because each of the three channels gets its own 8-bit byte (256 possible values), a single color pixel requires 3 bytes of memory.

Numerical Example: Let's look at the math behind "millions of colors". If Red, Green, and Blue each have 256 possible levels, the total number of color combinations is:

$$256 \times 256 \times 256 = 16,777,216 \text{ possible colors.}$$

For instance, a pixel with the triplet $(255, 0, 0)$ is telling the monitor to display maximum Red, zero Green, and zero Blue. A triplet of $(255, 255, 255)$ mixes all light to create pure white.

When processing these images algorithmically, engineers typically face a choice: either process the R, G, and B channels completely independently and merge the results at the end, or devise ad-hoc algorithms that analyze the complex interplay of the colors together.

2 Global Descriptors and Image Segmentation

2.1 The Goal: Image Segmentation

One of the most fundamental tasks in computational vision is **image segmentation**. This means dividing an image into meaningful parts, essentially detecting groups of pixels that are physically close together and visually similar.

We can evaluate "similarity" based on various attributes, such as:

- **Brightness** (in grayscale images)
- **Color**
- **Texture** or **Motion**

The simplest example is separating a foreground object (like the cameraman in the slides) from the background.

2.2 Brightness Histograms and Binarization

To figure out which pixels belong to the foreground and which belong to the background, we use a **brightness histogram**.

- **What is it?** A histogram is a graph where the x-axis represents the possible pixel values (0 to 255 for an 8-bit image) and the y-axis represents the number of pixels in the image that have that exact value.

If an image has a dark background and a bright subject, the histogram will typically show two distinct "peaks" (a bimodal histogram). We can achieve segmentation through **binarization**. This involves picking a threshold value directly between those two peaks.

Numerical Example: Imagine an image where the background pixels mostly have values around 30 (dark gray) and the object pixels are around 200 (light gray). We can set a threshold at $T = 100$. The algorithm looks at every pixel $I(x, y)$:

- If $I(x, y) < 100$, it sets the pixel to 0 (pure black).
- If $I(x, y) \geq 100$, it sets the pixel to 255 (pure white).

The result is a clean binary image (a silhouette).

2.3 The Challenge of Contrast

The slides point out a critical reality: finding that perfect threshold is rarely easy! The shape of the histogram tells us a lot about the image quality:

- **Dark image:** All pixels are bunched up on the left side (near 0).
- **Bright image:** All pixels are bunched up on the right side (near 255).
- **Low-contrast image:** The pixels are squished together in a narrow spike in the middle. It's very hard to separate objects here.
- **High-contrast image:** The pixels are spread widely across the entire 0-255 range. This is ideal for thresholding.

2.4 Dealing with Color

When we move to color images, segmentation gets trickier. The slides note that in color, values being numerically "close" in raw RGB code doesn't always mean they look visually "similar" to human eyes.

For example, a pixel in shadow might have entirely different RGB numbers than a sunlit pixel of the exact same object. Because of this, a careful choice of the **color space** (like moving from RGB to HSV - Hue, Saturation, Value) is highly recommended before trying to segment.

2.5 Image Quantization

Another technique mentioned is **Image Quantization**, which is the deliberate reduction of the image's dynamic range. Instead of having 256 shades of gray, we might force the image to only use 4 or 8 shades. This simplifies the image drastically, washing out minor noise and making it a very useful pre-processing step before attempting segmentation.

Numerical Example: Let's say we want to reduce 256 levels down to just 4 levels. We divide the range into 4 "buckets" of 64 values each. For a pixel with an original value of 150:

$$New_Value = \left\lfloor \frac{150}{64} \right\rfloor = 2$$

We map it to bucket "2", completely erasing the subtle differences between value 150 and value 160 (they both go into the same bucket).

2.6 Summary of Global Descriptors (Histograms)

Histograms are used to obtain an overall description of the image content.

- **The Good:** They are very easy to compute and highly robust to geometrical variations. If you take a picture of an apple and rotate the photo 90 degrees, the histogram remains exactly the same.
- **The Bad:** They completely **lose spatial information**.

Context: To understand why losing spatial information is bad, imagine an image of a chessboard. Now imagine a second image where the top half is solid black and the bottom half is solid white. Both images have 50% black pixels and 50% white pixels. Their histograms will be absolutely identical, even though the images look completely different!

3 Linear Filters and Local Features

3.1 Linear Filtering and Convolution

A basic tool in image processing is the linear filter, used for noise reduction and signal enhancement. Filtering is achieved through a mathematical operation called **convolution**.

In convolution, a small matrix called a **kernel** (k) is swept across the image signal (f) to produce a new filtered image (g):

$$g = f * k$$

For a 2D image, the math is the sum of the element-wise products between the kernel and the image patch it currently covers:

$$g[x, y] = \sum_{m,l} f[x - m, y - l]k[m, l]$$

Numerical Example: Imagine a 1D row of pixels transitioning from dark to light: $f = [10, 10, 10, 50, 50, 50]$. We want to find the edge using a simple edge-detection kernel: $k = [-1, 0, 1]$. Let's center the kernel over the third pixel (value 10):

$$g[3] = (10 \times -1) + (10 \times 0) + (50 \times 1) = -10 + 0 + 50 = 40$$

The high output value (40) indicates that a sharp transition (an edge) was detected at this location.

3.2 Image Gradients and Edges

Edges are pixels at which the image values undergo a sharp variation. To find them, we compute the **Image Gradient**, a vector that points in the direction of the most rapid change in intensity.

The gradient is composed of partial derivatives in the x and y directions:

$$\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

From this, we extract two crucial pieces of geometric information:

- **Gradient Magnitude** (Edge strength): $M(x, y) = \sqrt{g_x^2 + g_y^2}$
- **Gradient Orientation** (Edge direction): $\theta(x, y) = \arctan\left(\frac{g_y}{g_x}\right)$

3.3 Image Gradient: Numerical Example

To calculate the gradient at a specific pixel, we estimate the partial derivatives using the **central difference** of its immediate neighbors.

Imagine a 3×3 patch of grayscale image pixels, where we want to find the gradient of the center pixel (x, y):

$$\text{Patch} = \begin{bmatrix} 10 & 10 & 10 \\ 10 & \mathbf{50} & 90 \\ 10 & 90 & 170 \end{bmatrix}$$

3.3.1 Calculate Partial Derivatives

We find the rate of change in the horizontal (x) and vertical (y) directions. *(Note: In standard image coordinates, the y-axis increases downwards).*

- **Horizontal change (g_x):** Pixel to the Right - Pixel to the Left

$$g_x = f(x + 1, y) - f(x - 1, y) = 90 - 10 = 80$$

- **Vertical change (g_y):** Pixel Below - Pixel Above

$$g_y = f(x, y + 1) - f(x, y - 1) = 90 - 10 = 80$$

3.3.2 Calculate Gradient Magnitude

The magnitude measures the **strength** of the edge.

$$M(x, y) = \sqrt{g_x^2 + g_y^2}$$

$$M = \sqrt{80^2 + 80^2} = \sqrt{6400 + 6400} = \sqrt{12800} \approx 113.1$$

3.3.3 Calculate Gradient Orientation

The orientation measures the **direction** of the most rapid increase in intensity.

$$\theta(x, y) = \arctan\left(\frac{g_y}{g_x}\right)$$

$$\theta = \arctan\left(\frac{80}{80}\right) = \arctan(1) = 45^\circ$$

Conclusion: The gradient vector points at a 45° angle (down and to the right). The gradient always points in the direction of steepest ascent (from dark to bright).

3.4 The Effect of Noise and Gaussian Smoothing

Calculating derivatives is highly sensitive to noise. Taking the derivative of a noisy signal results in massive spikes that hide the true edges.

The Solution: We must smooth the image before taking the derivative. We typically use a **Gaussian Filter**, which acts as a low-pass filter to blur out random noise.

$$h_\sigma(u,v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$

Mathematical Shortcut: Because convolution is associative, taking the derivative of a smoothed image is mathematically identical to smoothing the image with the *derivative of the smoothing filter*:

$$\frac{\partial}{\partial x}(h * f) = \left(\frac{\partial}{\partial x} h\right) * f$$

This saves computational power by allowing us to use a single "Derivative of Gaussian" filter. This logic forms the foundation of the famous **Canny Edge Detector**.

3.5 Local Features: Corners

While edges are useful, they suffer from the aperture problem (you cannot tell where along an edge you are). **Corners** are patches with strong gradients in at least *two* significantly different orientations, making them stable and "good features to match" across different images.

3.5.1 The Autocorrelation Function

To detect corners, we analyze how a local patch of pixels changes when shifted by a small vector u. We use the Summed Square Difference (SSD), also known as the autocorrelation function $E_{AC}(u)$:

$$E_{AC}(u) = \sum_i [I(x_i + u) - I(x_i)]^2$$

To make this computationally feasible, we approximate the shifted image using a Taylor series expansion:

$$I(x_i + u) \approx I(x_i) + \nabla I(x_i) \cdot u$$

Substituting this back into the SSD equation cancels out the $I(x_i)$ terms, yielding a quadratic form:

$$E_{AC}(u) \approx \sum_i [\nabla I(x_i) \cdot u]^2 = u^\top A u$$

3.5.2 The Autocorrelation Matrix (A)

The matrix A captures the gradient distribution within the patch:

$$A = \begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$$

By calculating the eigenvalues (λ_1, λ_2) of A, we can classify the local geometry:

- **Corner:** λ_1 and λ_2 are both large.
- **Edge:** $\lambda_1 \gg \lambda_2$ or $\lambda_2 \gg \lambda_1$.
- **Flat Region:** Both λ_1 and λ_2 are small.

3.5.3 Corner Numerical Example

Consider a 4×4 image grid containing a dark square (intensity 10) on a light background (intensity 100). We will compute the Autocorrelation Matrix for the inner 2×2 patch.

$$\text{Image} = \begin{bmatrix} 100 & 100 & 100 & 100 \\ 100 & \mathbf{10} & \mathbf{10} & 100 \\ 100 & \mathbf{10} & \mathbf{10} & 100 \\ 100 & 100 & 100 & 100 \end{bmatrix}$$

Step 1: Calculate Central Difference Gradients (I_x, I_y) For the four inner pixels:

- **Top-Left:** $I_x = 10 - 100 = -90$, $I_y = 10 - 100 = -90$
- **Top-Right:** $I_x = 100 - 10 = 90$, $I_y = 10 - 100 = -90$
- **Bottom-Left:** $I_x = 10 - 100 = -90$, $I_y = 100 - 10 = 90$
- **Bottom-Right:** $I_x = 100 - 10 = 90$, $I_y = 100 - 10 = 90$

Step 2: Calculate Matrix Components Squaring and summing these values over the 4 pixels:

$$\sum I_x^2 = (-90)^2 + (90)^2 + (-90)^2 + (90)^2 = 32400$$

$$\sum I_y^2 = (-90)^2 + (-90)^2 + (90)^2 + (90)^2 = 32400$$

$$\sum I_x I_y = 8100 - 8100 - 8100 + 8100 = 0$$

Step 3: Construct Matrix A and Extract Eigenvalues

$$A = \begin{bmatrix} 32400 & 0 \\ 0 & 32400 \end{bmatrix}$$

Because A is a diagonal matrix, its eigenvalues are simply the diagonal entries:

$$\lambda_1 = 32400$$

$$\lambda_2 = 32400$$

Conclusion: Because both eigenvalues are exceedingly large, the algorithm confidently classifies this patch as a **Corner**.

4 Image Segmentation

Image segmentation is a fundamental task in computational vision that involves partitioning a digital image into multiple segments (sets of pixels, also known as image objects). The goal of segmentation is to simplify and/or change the representation of an image into something that is more meaningful and easier to analyze.

4.1 Core Concept: Uniformity

Segmentation typically divides an image into regions that are uniform according to a specific quality or feature:

- **Brightness/Intensity:** Common light levels.
- **Color:** Similarities in the color space.
- **Motion:** Pixels moving with the same velocity/direction.

4.2 Mathematical Definition

Let R represent the entire spatial region occupied by an image. Image segmentation is the process of partitioning R into n subregions $R_1, R_2, \ldots, R_n$ such that:

1. **Completeness:** $\bigcup_{i=1}^{n} R_i = R$. Every pixel must belong to a region.
2. **Connectivity:** R_i is a connected set for $i = 1, \ldots, n$.
3. **Disjointness:** $R_i \cap R_j = \emptyset$ for all $i \neq j$. Regions cannot overlap.
4. **Uniformity Predicate:** $Q(R_i) = \text{TRUE}$ for $i = 1, \ldots, n$. All pixels in a region satisfy a logical predicate Q (e.g., same intensity).
5. **Maximality:** $Q(R_i \cup R_j) = \text{FALSE}$ for any adjacent regions R_i and R_j. Neighboring regions must be different according to the predicate Q.

4.3 Methods Overview

Segmentation methods can be broadly categorized based on the level of guidance and the visual cues used.

4.3.1 Supervision Levels

- **Unsupervised:** Segmentation is based purely on pixel and appearance information (low-level features).
- **Supervised:** Segmentation is guided by semantic concepts or prior knowledge of specific object classes.

4.3.2 Technical Approaches

- **Edge-based methods:** Look for discontinuities (edges) in the image to find boundaries between regions.
- **Region-based methods:** Look for regions of similarity according to specific criteria. This is generally considered a more robust formulation.

5 Thresholding and Motion Segmentation

This chapter focuses on **Thresholding**, a region-based segmentation method that groups pixels sharing similar intensity levels.

5.1 Basic Thresholding Concepts

The simplest form of segmentation is the binary case, where we separate an image into two classes: Foreground and Background. This is achieved by comparing each pixel intensity $f(x, y)$ to a global threshold T.

5.1.1 Global Binary Thresholding

The resulting image $g(x, y)$ is defined as:

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

Where 1 typically represents the foreground and 0 the background.

5.1.2 Multiple Thresholding

When an image contains multiple objects with distinct intensity ranges, a single threshold is insufficient. Multiple thresholds $(T_1, T_2, \dots)$ can be used:

$$g(x,y) = \begin{cases} a & \text{if } f(x,y) > T_2 \\ b & \text{if } T_1 < f(x,y) \leq T_2 \\ c & \text{otherwise} \end{cases} \tag{2}$$

5.2 Motion Segmentation

Motion segmentation is a practical application of thresholding where the "uniformity" criterion is motion rather than static intensity. By identifying pixels that change significantly between frames, we can extract moving objects from a static background. This is a binary segmentation task (Moving vs. Not Moving).

5.3 Automatic Global Threshold Selection

Instead of manually picking T, we can use an iterative algorithm to find the optimal value, typically located in the "valley" of a bimodal histogram.

5.3.1 The Iterative Algorithm

1. **Initial Estimate:** Select an initial T (often the global mean intensity of the image).
2. **Segmentation:** Partition the image into two groups: G_1 (pixels $> T$) and G_2 (pixels $\leq T$).
3. **Mean Computation:** Calculate the average intensity values m_1 and m_2 for groups G_1 and G_2 respectively.
4. **Update:** Compute a new threshold $T = \frac{1}{2}(m_1 + m_2)$.
5. **Iteration:** Repeat steps 2 through 4 until T stabilizes (the change is smaller than a predefined epsilon).

5.3.2 Numerical Example

Suppose we have a set of 16 pixel intensities:

$$\{10, 12, 15, 20, 22, 25, 100, 110, 120, 130, 140, 150, 160, 170, 180, 200\}$$

Iteration 1:

- **Initial T_0:** Global mean ≈ 101.
- **Groups:** $G_1 = \{110, \dots, 200\}$ (9 pixels), $G_2 = \{10, \dots, 100\}$ (7 pixels).
- **Means:** $m_1 = 151.1$, $m_2 = 29.1$.
- **New T:** $\frac{1}{2}(151.1 + 29.1) = 90.1$.

Iteration 2:

- **Thresholding with $T = 90.1$:** The value 100 now moves from G_2 to G_1.
- **New Groups:** $G_1 = \{100, 110, \dots, 200\}$ (10 pixels), $G_2 = \{10, \dots, 25\}$ (6 pixels).
- **New Means:** $m_1 = 146$, $m_2 = 17.3$.
- **New T:** $\frac{1}{2}(146 + 17.3) = 81.65$.

The algorithm continues until the threshold value converges, effectively separating the dark cluster from the light cluster.

6 Otsu's Method for Global Thresholding

While iterative methods rely on heuristic averaging, Otsu's method is grounded in statistical discriminant analysis. Its objective is to segment foreground and background by maximizing the **between-class variance**. Computations are performed on the normalized image histogram.

6.1 Mathematical Formulation

Let the image have intensity levels from 0 to $L-1$. The normalized histogram gives the probability p_i of each intensity level i.

We define a threshold k $(0 < k < L-1)$ that divides pixels into two classes:

- C_1: Pixels with intensities $[0, k]$
- C_2: Pixels with intensities $[k+1, L-1]$

The probabilities of a pixel belonging to C_1 or C_2 are:

$$P_1(k) = \sum_{i=0}^{k} p_i \tag{3}$$

$$P_2(k) = \sum_{i=k+1}^{L-1} p_i = 1 - P_1(k) \tag{4}$$

The mean intensities of the classes are:

$$m_1(k) = \frac{1}{P_1(k)} \sum_{i=0}^{k} i p_i \tag{5}$$

$$m_2(k) = \frac{1}{P_2(k)} \sum_{i=k+1}^{L-1} i p_i \tag{6}$$

The global mean of the entire image is $m_G = \sum_{i=0}^{L-1} i p_i$.

6.2 Optimization Criterion

Otsu's method finds the optimal threshold k^* that maximizes the between-class variance $\sigma_B^2(k)$:

$$k^* = \arg \max_{0 \le k \le L-1} \sigma_B^2(k) \tag{7}$$

Where the between-class variance is defined as:

$$\sigma_B^2(k) = P_1(k)(m_1(k) - m_G)^2 + P_2(k)(m_2(k) - m_G)^2 \tag{8}$$

6.3 Numerical Example

Consider a 10-pixel image with 4 intensity levels ($L = 4$). Pixel values: $\{0, 0, 0, 1, 1, 2, 3, 3, 3, 3\}$.

Probabilities: $p_0 = 0.3$, $p_1 = 0.2$, $p_2 = 0.1$, $p_3 = 0.4$. Global Mean: $m_G = 0(0.3) + 1(0.2) + 2(0.1) + 3(0.4) = 1.6$.

We evaluate the variance for potential thresholds k:

- **For $k = 0$:** $P_1 = 0.3, P_2 = 0.7, m_1 = 0, m_2 \approx 2.28$.
  $\sigma_B^2(0) = 0.3(0 - 1.6)^2 + 0.7(2.28 - 1.6)^2 \approx 1.09$.
- **For $k = 1$:** $P_1 = 0.5, P_2 = 0.5, m_1 = 0.4, m_2 = 2.8$.
  $\sigma_B^2(1) = 0.5(0.4 - 1.6)^2 + 0.5(2.8 - 1.6)^2 = 1.44$.
- **For $k = 2$:** $P_1 = 0.6, P_2 = 0.4, m_1 \approx 0.67, m_2 = 3$.
  $\sigma_B^2(2) = 0.6(0.67 - 1.6)^2 + 0.4(3 - 1.6)^2 \approx 1.30$.

The maximum variance is 1.44 at $k = 1$. Therefore, the optimal threshold is $T = 1$.

7 Improving Thresholding

Global thresholding can fail if the image is noisy or if lighting is uneven.

7.1 Image Smoothing

Noisy images produce jagged histograms where it is difficult to find a clear valley. Applying Gaussian smoothing prior to thresholding smooths the histogram, making global thresholding algorithms (like Otsu's) significantly more effective.

7.2 Variable (Local) Thresholding

When an image has uneven illumination (e.g., shadows across a document), a single global threshold will fail. Variable thresholding solves this by subdividing the image or computing a unique threshold T_{xy} for every pixel based on its local neighborhood.

A common approach computes the threshold based on local mean (m_{xy}) and standard deviation (σ_{xy}):

$$T_{xy} = a\sigma_{xy} + bm_{xy} \quad (a, b \geq 0) \tag{9}$$

For text segmentation, a **moving average** approach is highly effective. It calculates the threshold $T_{xy} = bm_{xy}$ where m is estimated using only the last n visited pixels along a scan line.